PYTHON HANDBOOK

10-page visual study notes • Beginner to OOP foundations

01 INTRODUCTION TO PYTHON

What is Python? Python is a high-level, general-purpose programming language known for readable syntax. It is used for automation, web development, data analysis, AI, and many other tasks.

WHY PYTHON?

• Readable syntax that is friendly for beginners.

• Interpreted: code is executed by a Python interpreter.

• Dynamically typed: variable types are determined at runtime.

• Cross-platform: works on Windows, macOS, and Linux.

• Large standard library and a wide ecosystem of packages.

A FIRST PROGRAM

```
print("Hello, World!")
```

CORE IDEAS

Term	Meaning
Program	Instructions that tell a computer what to do.
Syntax	Rules for writing valid code.
Bug	A mistake that makes code fail or behave incorrectly.
Comment	A note for humans; Python ignores it.

```
# This is a comment
print("Python is fun!")
```

Remember: Python is case-sensitive. `print` and `Print` are different names.

02 VARIABLES, TYPES & OPERATORS

Think of a variable as a label that refers to a value.

VARIABLES

```
name = "Faizan"
age = 16
is_learning = True
height = 1.72
```

Rules: use meaningful names; begin with a letter or underscore; do not start with a digit; avoid Python keywords; use snake_case such as `total_marks`.

BUILT-IN DATA TYPES

Type	Example	Use
int	42	Whole numbers
float	3.14	Decimal values
str	"hello"	Text
bool	True / False	Logical values
NoneType	None	No value / absence of value

TYPE CONVERSION

```
age_text = "16"
age_number = int(age_text)
price = float("12.50")
message = str(100)
print(type(age_number))
```

OPERATORS

Arithmetic: +, -, *, /, // (floor division), % (remainder), ** (power).

Comparison: ==, !=, >, <, >=, <= return True or False.

Logical: and, or, not. **Assignment:** =, +=, -=, *=.

```
print(7 // 2)  # 3
print(7 % 2)   # 1
print(2 ** 3)  # 8
```

input() returns text. Convert it when you need a number: `age = int(input("Age: "))`.

03 STRINGS

A string is text surrounded by quotes.

CREATING STRINGS

```
first = "Python"
second = 'is useful'
multiline = """Line one
Line two"""
```

INDEXING & SLICING

```
word = "PYTHON"
print(word[0])    # P
print(word[-1])   # N
print(word[1:4])  # YTH
print(word[:3])   # PYT
print(word[::2])  # PTO
```

The first index is 0. The slice end index is excluded. Strings are **immutable**: you cannot replace one character directly.

USEFUL STRING METHODS

```
text = "  Learn Python  "
text.strip()       # remove outer whitespace
text.lower()       # lowercase
text.upper()       # uppercase
text.title()       # Title Case
text.replace("Python", "coding")
"one,two".split(",")
"-".join(["a", "b"])
```

CHECKING & FORMATTING

```
"py" in "python"        # True
"123".isdigit()         # True
"hello".startswith("he")
name = "Faizan"
score = 92
print(f"{name} scored {score}")
```

String methods return results; most do not change the original string. Since strings are immutable, assign the result if you want to keep it.

MINI PRACTICE

1. Ask for a name and print it in title case. 2. Count how many characters are in a sentence. 3. Replace spaces with hyphens.

04 LISTS

A list stores an ordered, changeable collection. Duplicates are allowed.

CREATING & ACCESSING

```
fruits = ["apple", "mango", "banana"]
print(fruits[0])
print(fruits[-1])
print(fruits[1:3])
```

CHANGE. ADD. REMOVE

```
fruits[1] = "orange"
fruits.append("grape")       # add one at end
fruits.extend(["pear", "kiwi"])
fruits.insert(1, "mango")    # insert at index
fruits.remove("apple")       # remove by value
last = fruits.pop()          # remove and return last
del fruits[0]               #delete by index
```

COMMON LIST TOOLS

```
numbers = [4, 1, 8, 2]
len(numbers)       # 4
min(numbers)       # 1
max(numbers)       # 8
sum(numbers)       # 15
numbers.sort()     # sorts in place
numbers.reverse()  # reverses in place
copy_numbers = numbers.copy()
```

LOOPING THROUGH A LIST

```
for fruit in fruits:
    print(fruit)

for index, fruit in enumerate(fruits):
    print(index, fruit)
```

IMPORTANT

• Lists are mutable: items can be changed. • `sort()` changes the same list and returns None. • `sorted(numbers)` creates a new sorted list. • Use `in` to check membership.

List vs variable: a list can hold many values under one name and can grow or shrink.

MINI PRACTICE

Create a list of five marks, add one mark, remove one mark, then print the highest and average.

05 TUPLES, SETS & DICTIONARIES

Choose a collection based on how you need to store and find data.

TUPLES — ORDERED, IMMUTABLE

```
point = (4, 7)
colors = ("pink", "blue", "green")
single_item = (5,)  # comma makes it a tuple
print(point[0])
x, y = point  # unpacking
```

Use a tuple for a group of values that should not be changed, such as coordinates.

SETS — UNIQUE ITEMS, NO INDEXING

```
skills = {"Python", "SQL", "Python"}
print(skills)  # duplicate removed
skills.add("Git")
skills.discard("SQL")
A = {1, 2, 3}
B = {3, 4, 5}
A | B  # union: {1,2,3,4,5}
A & B  # intersection: {3}
A - B  # difference: {1,2}
```

DICTIONARIES — KEY : VALUE

```
student = {"name": "Faizan", "age": 16}
print(student["name"])
print(student.get("grade", "Not set"))
student["grade"] = "A"       # add/update
student.update({"age": 17})
student.pop("age")
print(student.keys())
print(student.values())
print(student.items())
```

Loop through dictionary

```
for key, value in student.items():
    print(key, value)
```

QUICK COMPARISON

Type	Ordered?	Mutable?	Duplicates?
list	Yes	Yes	Yes
tuple	Yes	No	Yes
set	No indexing	Yes	No
dict	Insertion order	Yes	Keys unique

Dictionary keys must be hashable (commonly strings, numbers, or tuples of hashable values).

06 CONDITIONS, LOOPS & LOGIC

Control flow lets your program make decisions and repeat work.

IF / ELIF / ELSE

```
marks = 78
if marks >= 80:
    grade = "A"
elif marks >= 70:
    grade = "B"
else:
    grade = "Keep practicing"
print(grade)
```

Indentation matters. Python uses indentation to define a block.

FOR LOOP

```
for number in range(1, 6):
    print(number)  # 1 to 5

for letter in "code":
    print(letter)
```

`range(start, stop, step)` excludes the stop value. Example: `range(2, 10, 2)` produces 2, 4, 6, 8.

WHILE LOOP

```
count = 1
while count <= 3:
    print(count)
    count += 1
```

A while loop needs a condition that eventually becomes false, or it may run forever.

BREAK, CONTINUE & LOOP ELSE

```
for n in range(1, 6):
    if n == 3:
        continue  # skip 3
    if n == 5:
        break     # stop loop
    print(n)
```

LOGIC OPERATORS

and: all conditions must be true. **or**: at least one is true. **not**: reverses a truth value. Use parentheses to make complex conditions easier to read.

```
age = 17
has_ticket = True
if age >= 16 and has_ticket:
    print("Allowed")
```

Common beginner bugs: using = instead of == in a comparison; forgetting a colon; incorrect indentation; infinite while loops.

MINI PRACTICE

Print numbers 1–30; label each as even or odd. Then write a number-guessing loop that stops when the guess is correct.

07 FUNCTIONS, METHODS & SCOPE

A function packages reusable work. A method is a function accessed through an object or class.

FUNCTION BASICS

```
def greet(name):
    return f"Hello, {name}!"

message = greet("Faizan")
print(message)
```

def defines a function. **name** is a parameter. **"Faizan"** is an argument. **return** sends a result back to the caller.

PARAMETERS & ARGUMENTS

```
def power(base, exponent=2):
    return base ** exponent

print(power(3))       # 9
print(power(2, 3))    # 8
print(power(base=4, exponent=3))
```

RETURN VS PRINT

print() displays a value for a person to see. **return** gives a value back to the code that called the function, so it can be stored or used in calculations.

```
def add(a, b):
    return a + b

result = add(2, 3)  # result is 5
```

FUNCTION VS METHOD

```
def shout(text):          # function
    return text.upper()

word = "hello"
print(shout(word))
print(word.upper())       # str method
```

A method is called using dot notation, like `word.upper()` or `my_list.append(4)`. Methods often work with the object they are attached to.

SCOPE & COMMON PATTERNS

```
def calculate_total(prices):
    total = sum(prices)  # local variable
    return total

print(calculate_total([10, 20, 5]))
```

Variables created inside a function are usually local to it. Prefer parameters and return values instead of relying on global variables.

LAMBDA & DOCSTRINGS

```
square = lambda x: x * x
print(square(5))

def area(width, height):
    """Return the area of a rectangle."""
    return width * height
```

08 OOP: CLASSES, OBJECTS & SELF

Object-oriented programming groups data and related behavior together.

CLASS VS OBJECT

A **class** is a blueprint. An **object** is one instance created from that blueprint. Each object can hold its own attribute values.

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def introduce(self):
        print(f"I am {self.name}, age {self.age}.")

student1 = Student("Ali", 16)
student2 = Student("Sara", 17)
student1.introduce()
student2.introduce()
```

UNDERSTANDING SELF

`self` refers to the current instance. When you call `student1.introduce()`, Python passes `student1` as `self` automatically. Thus `self.name` means the name stored on that particular object.

__init__

`__init__` is an initializer that runs when an instance is created. It sets up the object's starting state; it is not the same as creating the class.

ATTRIBUTES & METHODS

```
class Dog:
    species = "Canine"  # class attribute

    def __init__(self, name):
        self.name = name  # instance attribute

    def bark(self):
        return f"{self.name} says woof!"

dog = Dog("Milo")
print(dog.name)
print(dog.bark())
```

Instance attributes belong to each object; class attributes are shared through the class unless shadowed by an instance attribute.

WHY OOP?

OOP helps organize larger programs, model real-world entities, keep related data and behavior together, and make code easier to extend. Functions can also solve many problems; OOP is useful when objects with state and behavior make the design clearer.

Mental model: **class** = blueprint; **object** = actual thing; **attribute** = data; **method** = behavior.

09 OOP: FOUR PILLARS & RELATIONSHIPS

These ideas help you design code that can grow without becoming tangled.

1. ENCAPSULATION

Keep data and methods together, and control how an object's state is accessed or changed. Python uses conventions such as `_name` for internal use and name mangling with `__name`; these are not absolute security barriers.

```
class BankAccount:
    def __init__(self, balance=0):
        self._balance = balance

    def deposit(self, amount):
        if amount > 0:
            self._balance += amount

    def get_balance(self):
        return self._balance
```

2. INHERITANCE

A child class can reuse or specialize behavior from a parent class. Use inheritance for a genuine “is-a” relationship.

```
class Person:
    def __init__(self, name):
        self.name = name

class Doctor(Person):
    def diagnose(self):
        return f"Dr. {self.name} is examining a patient."
```

3. POLYMORPHISM

Different objects can respond to the same method name in their own way. Python commonly uses duck typing: an object is useful if it supports the needed behavior.

```
class Cat:
    def speak(self): return "Meow"
class Cow:
    def speak(self): return "Moo"

for animal in [Cat(), Cow()]:
    print(animal.speak())
```

4. ABSTRACTION

Expose the essential interface while hiding implementation details. Python's `abc` module can define abstract base classes when a shared interface must be implemented.

COMPOSITION VS INHERITANCE

Composition means one object contains or uses another object (“has-a”). For example, a Hospital has doctors. Inheritance means one class is a specialized kind of another (“is-a”). Prefer composition when it models the relationship more naturally.

SPECIAL METHODS

```
class Book:
    def __init__(self, title):
        self.title = title
    def __str__(self):
        return self.title

print(Book("Python Basics"))
```

Other useful methods include `__repr__`, `__len__` and operator methods such as `__add__`.

10 ERRORS, FILES, MODULES & PRACTICE

Put the pieces together and build a habit of testing small steps.

ERROR HANDLING

```
try:
    number = int(input("Enter a number: "))
    print(100 / number)
except ValueError:
    print("Please enter a whole number.")
except ZeroDivisionError:
    print("Cannot divide by zero.")
finally:
    print("Finished.")
```

Catch errors you can handle specifically; avoid a bare `except:` unless you have a clear reason.

FILES

```
with open("notes.txt", "w", encoding="utf-8") as file:
    file.write("Learning Python")

with open("notes.txt", "r", encoding="utf-8") as file:
    content = file.read()
    print(content)
```

Use `with` so the file is properly closed. Common modes: `r` read, `w` write/overwrite, `a` append.

MODULES & PACKAGES

```
import math
print(math.sqrt(25))

from random import randint
print(randint(1, 10))
```

A module is a Python file you can import. A package organizes modules. Install third-party packages with pip in the correct environment.

DEBUGGING CHECKLIST

- Read the last line of the traceback first, then inspect the relevant line.
- Check spelling, capitalization, parentheses, colons, and indentation.
- Print or inspect intermediate values; use small test inputs.
- Separate a large task into functions with clear names.
- Test normal cases, edge cases, and invalid input.

10-PAGE LEARNING CHECKLIST

- Explain list vs tuple vs set vs dictionary without memorizing.
- Write functions with parameters and return values.
- Explain function vs method, argument vs parameter, class vs object.
- Create a class with __init__, attributes, and instance methods.
- Demonstrate inheritance, encapsulation, polymorphism, and abstraction.
- Build a small CRUD project using a dictionary of objects.

PROJECT IDEA: HOSPITAL MANAGEMENT

Create `Doctor` objects, then store them in `hospital.doctors` using doctor ID as the dictionary key and the Doctor object as the value. Implement add, view, update, and delete methods; check whether an ID exists before updating or deleting.

Learn by writing and testing code, not only by reading notes. Change examples, predict the output, then run them.